

TECHNICAL ROADMAP TEMPLATE



Submission Deadline: May 17, 2026

Format: PDF (Strictly maximum 5 pages)

Naming Convention: [Track]_[University]_[TeamName]_Road map.pdf

TEAM INFORMATION

Team Name	DynaVation
Institution	Davao del Sur State College (DSSC)
Country	Philippines
Track	Climate Change
Team Leader Name	John Rey R. Sto. Domingo, johnrey.sto.domingo@dssc.edu.ph , +639061420540

SECTION 1: EXECUTIVE SUMMARY (PROBLEM-SOLUTION FIT)

Flooding remains one of the most devastating climate-related hazards in the Philippines and across vulnerable ASEAN communities, causing displacement, delayed evacuations, infrastructure damage, and loss of life every year. As climate change intensifies extreme rainfall and rising water levels, many local communities still rely on costly rescue equipment, fragmented communication systems, and manual coordination processes that slow emergency response operations during critical situations. Existing evacuation approaches often lack real-time situational awareness, intelligent route coordination, and accessible technologies that can support both rescuers and affected households during rapidly changing flood conditions.

To address these challenges, this project proposes an integrated AI-driven flood response ecosystem designed to strengthen community resilience through affordable, data-driven, and real-time emergency coordination. The framework combines IoT-assisted emergency boats built from recyclable plastic materials, GPS-enabled monitoring, AI-supported risk analysis, and real-time communication technologies within a unified disaster response platform. Through mobile and web-based systems, the solution enables real-time tracking of evacuation assets, intelligent route optimization, occupancy verification, hazard monitoring, and coordinated rescue management for households, responders, barangays, and local government units. By integrating artificial intelligence, predictive analytics, and low-cost sustainable technologies, the proposed framework aims to improve evacuation efficiency, enhance operational safety, reduce response delays, and support climate-resilient communities in disaster-prone regions.

SECTION 2: TECHNICAL ARCHITECTURE

2.1 System Components:

Inputs: The system receives real-time telemetry from IoT-enabled emergency boats, including GPS location (lat/lon), water level (cm), boat occupancy count (passengers), battery level (%), stability/motion data (accelerometer), and camera feed for passenger verification. Optional inputs include weather API data (rainfall intensity, wind speed), flood hazard maps,

and manual rescue requests from the mobile app or barangay web dashboard. The system can also receive voice commands via offline speech recognition on the boat's edge device.

Processing Core: AI processing occurs at two layers: (1) Edge AI on the boat (ESP32 or Raspberry Pi) for real-time functions — passenger detection (lightweight CNN), stability monitoring (accelerometer thresholds), battery life prediction (regression model), and voice command recognition (offline speech-to-text). (2) Cloud/Server AI for route optimization — Dijkstra's Algorithm computes safer and faster navigation paths based on real-time hazard data, weather, and water levels. Prescriptive analytics provide actionable recommendations (e.g., "Boat 3 reassign to Barangay 7"). A Flask/FastAPI backend (port 5000) orchestrates data between edge devices, mobile app, and web dashboard. PostgreSQL stores historical disaster records, boat registrations, and evacuation logs.

Outputs: The system delivers real-time boat locations on a flood-risk map, passenger occupancy alerts, stability warnings (e.g., "Unstable — reduce passengers"), battery low alerts, optimized evacuation routes, auto-assigned rescuer notifications, QR-based boat ownership verification, and push notifications via mobile app. A web dashboard for barangay/municipal administrators displays live lifeboat status, flood-prone area heatmaps, historical disaster analytics, and command acknowledgment tracking (ACK/ERROR).

2.2 Architecture Diagram:

TECHNICAL ARCHITECTURE & FLOW - DYNAVATION FLOOD RESPONSE

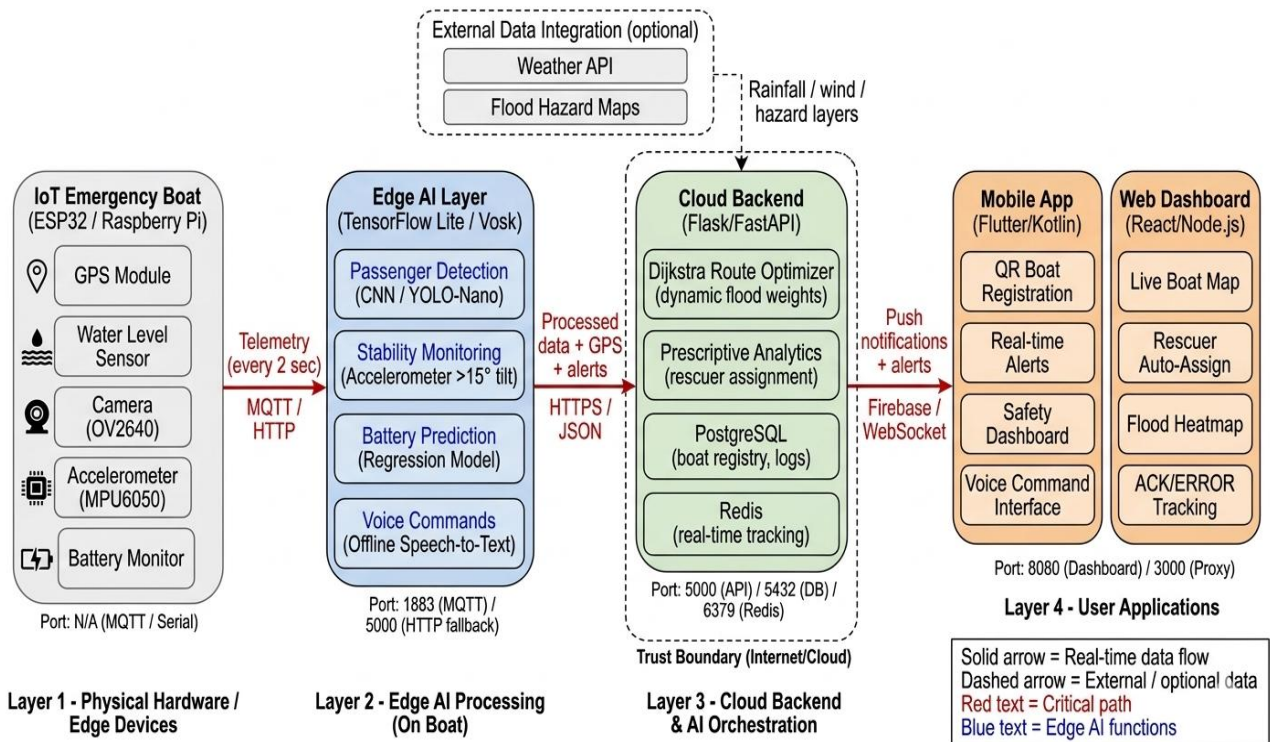


Figure 1. DynaVation system architecture: IoT emergency boat → Edge AI processing → Cloud backend (Dijkstra routing, prescriptive analytics) → Mobile app & Web dashboard for community and barangay users.

Architecture details derived from the source code and docs:

- IoT Boat Hardware: GPS module, water level sensor, camera (OV2640), accelerometer (MPU6050), battery monitor, and ESP32 or Raspberry Pi Zero for edge AI.
- Edge AI Layer (on boat): Passenger detection uses lightweight CNN, stability monitoring uses accelerometer threshold (tilt > 15° = warning), battery prediction uses linear regression, and voice commands use offline speech-to-text.
- Cloud/Backend Layer: Dijkstra's Algorithm computes route optimization with dynamic edge weights (flood depth, current speed), and prescriptive analytics uses priority scoring for boat assignment (population density + vulnerability).
- Mobile Application Layer: QR code enables household registration and boat ownership verification, real-time push notifications deliver flood warnings and boat arrival time, and safety dashboard displays battery, stability, and occupancy.

- Web Dashboard Layer: Barangay/municipal view shows live boat map, rescuer auto-assignment, and historical disaster trends, while command center provides ACK/ERROR tracking and manual override.
- External Data Integration: Weather API provides rainfall and wind data, with optional flood hazard map layers from OpenStreetMap or government data..

Core non-negotiable system flow: IoT Boat Sensors → Edge AI (on boat) → Cloud Backend (Dijkstra + prescriptive) → Mobile App / Web Dashboard → Rescue Action ACK.

SECTION 3: AI APPROACH & MODEL SELECTION

Primary AI Approach: [X] Machine Learning [X] Computer Vision [] NLP [] Generative AI [X]

Other: Rule-based stability monitoring + Dijkstra's Algorithm for route optimization

Model Selection: The system uses multiple lightweight AI models suitable for edge deployment on ESP32 or Raspberry Pi. For passenger detection, YOLO-Nano or TensorFlow Lite CNN runs on the boat's edge device. For battery prediction, a linear regression model estimates remaining minutes of operation based on historical discharge rates. For voice commands, offline speech-to-text (Vosk or Picovoice) enables hands-free operation without internet connectivity. For stability monitoring, the system uses rule-based thresholds (tilt > 15° = warning) rather than a learned model to ensure deterministic safety responses. For route optimization, Dijkstra's Algorithm computes safer and faster navigation paths using dynamic edge weights based on real-time flood depth and current speed. Core frameworks already planned include Python, Flask/FastAPI, TensorFlow Lite, MQTT, PostgreSQL, and Firebase for push notifications.

Reasoning: This hybrid approach is the most efficient because life-safety flood response needs determinism for critical warnings, low latency for real-time boat telemetry, and graceful degradation when internet is unavailable. The rule-based stability monitoring already triggers warnings at tilt > 15°, while edge AI handles passenger counting and battery prediction locally. If cloud AI (prescriptive analytics for rescuer assignment) is unavailable, the system still performs critical safety functions: passenger detection, stability warnings, and local route guidance continue on the boat's edge device. When cloud connectivity is available, Dijkstra routing and prescriptive analytics improve evacuation coordination and rescue efficiency without becoming a single point of failure.

- Implemented warning thresholds: water level > 50 cm, tilt > 15°, battery < 30%, occupancy > 80% of boat capacity.
- Implemented critical thresholds: water level > 100 cm, tilt > 25°, battery < 10%, occupancy = 100% (overcapacity).
- Critical recommended actions defined in the system logic include: SEND_ALERT_TO_BARANGAY, NOTIFY_HOUSEHOLDS_VIA_APP, REASSIGN_RESCUER, EVACUATE_ZONE, and CUT_BOAT_ENGINE_IF_UNSTABLE. In the current MVP, hardware execution is partially simulated, but the mobile app and web dashboard display all alerts with ACK/ERROR tracking.
- Implemented API endpoints (planned): /boat/telemetry, /boat/status, /route/optimize, /rescue/assign, /alerts/push, /commands/ack, /dashboard/map, /analytics/historical, and /ai/predict_battery.

SECTION 4: DATA STRATEGY & ETHICS

(Based on Workshop 2: Technical Architecture and Data Ethics)

Data Sources: The MVP uses three primary data sources: (1) real-time telemetry from IoT boat sensors (GPS, water level, camera, accelerometer, battery), (2) optional weather API data (rainfall, wind), and (3) optional flood hazard map layers. All data is operational telemetry only: location, water depth, passenger count, tilt angle, battery percentage, timestamps, boat ID, and zone ID. No PII is collected.

Data Quality and Cleaning: Data quality is enforced through sensor calibration, timestamp metadata, unit standardization (cm, degrees, %), edge-based outlier filtering, and idempotent

database upserts. Test gates require GPS lock within 10 seconds, occupancy accuracy > 90%, and mobile alert delivery within 3 seconds.

Licensing and Legality: The software stack uses open technologies: Python, TensorFlow Lite, Flask, PostgreSQL, MQTT, and Firebase. The MVP runs on synthetic telemetry from flood drills, avoiding restricted private datasets. Weather API usage complies with free-tier licensing.

Bias Mitigation and Fairness: The MVP avoids PII entirely — camera counts passengers without facial recognition. Fairness risks come from unequal boat distribution across barangays and sensor placement differences. Mitigation includes transparent thresholds, human-in-the-loop for rescue assignments, and audit trails for every command.

- No PII in MVP; camera counts passengers without storing or recognizing faces.
- Rule-first safety design: stability warnings (tilt > 15°) trigger deterministically without AI.
- Audit trail mandatory: who, when, why, and result (ACK/ERROR).

Sensitive actions require human oversight: boat reassignment and evacuation orders need barangay operator approval.

SECTION 5: DEVELOPMENT MILESTONES (AGILE ROADMAP)

(Based on Workshop 3: Agile Prototyping)

The roadmap below is aligned to the actual repository status and the hackathon objective of preparing a stable semi-final prototype.

Phase	Activity/Task	Tools Used	Expected Outcomes
Sprint 1 (Completed / Current Baseline)	Establish IoT boat hardware foundation: ESP32 with GPS, water level sensor, camera, accelerometer, battery monitor. Edge AI setup for passenger detection (YOLO-Nano) and stability monitoring.	ESP32, Raspberry Pi, Arduino IDE, TensorFlow Lite, Python	MVP baseline with working boat telemetry and edge-based passenger counting.
Sprint 2	Build cloud backend: Flask/FastAPI server, Dijkstra route optimization with dynamic flood weights, PostgreSQL database for boat registry and evacuation logs, Redis for real-time tracking.	Flask/FastAPI, PostgreSQL, Redis, Docker, Dijkstra Algorithm	Reliable route calculation within 3 seconds and database storage of all telemetry.
Sprint 3	Develop mobile app and web dashboard: QR code registration for households, real-time push alerts (flood warning, boat arrival), live boat map, rescuer auto-assignment, ACK/ERROR tracking.	Flutter/Kotlin, React/Node.js, Firebase, Leaflet/OpenStreetMap	Demonstrable closed-loop response: boat telemetry → cloud routing → mobile alert → rescuer ACK → audit trail.
Sprint 4	Production-readiness for hackathon judging: integrate all components, offline mode testing (edge AI continues without internet), refine documentation, KPI scorecard, test report, and replayable 3-5 minute demo.	README_HACKATHON.md, ARCHITECTURE.md, KPI_SCORECARD.md, DEMO_SCRIPT.md, TEST_REPORT.md	Judge-ready prototype with reproducible setup, stable storyline, and measurable impact metrics (evacuation time reduction, occupancy accuracy > 90%).

SECTION 6: SCALABILITY & REGIONAL RESILIENCE

Scaling Potential: The architecture already separates edge AI, cloud backend, mobile app, and web dashboard layers. This makes it easy to add more boats, barangays, or municipalities as new zones without rewriting the whole system. The cloud backend provides a unified interface for boat registry and real-time tracking, while Dijkstra's Algorithm can compute routes consistently across multi-zone deployments. The same pattern can extend from one barangay in Davao del Sur to adjacent flood-prone municipalities, provincial-level disaster response systems, and regional deployments across ASEAN countries facing seasonal flooding and climate-driven displacement.

Technical Constraints: The current prototype is still an MVP. It uses simulated flood scenarios for default testing, lacks full production security and high availability, depends on internet connectivity for cloud routing (though edge AI continues offline), and has limited long-term historical storage and analytics. AI-based passenger detection accuracy depends on camera quality and lighting conditions, which must be validated separately in field deployment. Hardware durability in floodwaters, MQTT infrastructure reliability, and regional device standards must be validated during pilot testing.

- Regional deployment path: 1 boat → 1 barangay → 1 municipality → 1 province → multi-province network.
- Multi-device support already planned: GPS, water level sensor, camera, accelerometer, battery monitor, and optional rain gauge or current speed sensor.
- Clear extension path for multi-channel alerts: Mobile app push notifications, SMS for non-smartphone users, barangay PA system integration, and email for municipal administrators.
- Future resilience features: historical flood analytics, replay timeline of past evacuations, predictive flood modeling (water level forecasting), richer policy control for rescuer assignment, and additional sensors such as rain intensity, water current speed, and temperature.

INSTRUCTIONS FOR SUBMISSION:

1. **Strictly PDF:** Convert your final document to PDF.
2. **Visuals Matter:** Use clear diagrams for Section 2.
3. **Submission Link:** Team Leaders must upload this file to the **Official Submission Folder** [SUBMISSION - TECHNICAL ROADMAP](#) by **May 17, 2026** (9:00 PM GMT+7).